

발표 준비 노트 — 본인 파트 (06. CI/CD 오케스트레이션)

Talking Points

이 문서 목적: 발표 본번 + Q&A 대비 핵심 멘트 모음. 특히 "본인이 직접 짜지 않은 영역" 을 어떻게 정직하면서도 본인 기여로 framing 할지에 집중.

작성 시점: 2026-05-03 PPT 작업 중 작성, 발표 직전까지 가다듬음.

사용 방법: 발표 전날 저녁 + 발표 당일 아침 정독. Q&A 들어왔을 때 이 노트 멘트 그대로 응용.

1. Terraform 기여 framing — 가장 신경 써야 할 부분 ★

1-1. 사실 확인 (정직하게)

본인이 직접 **owned** 한 것:

- ✓ Jenkinsfile 안의 `terraform apply -var-file=...` 호출 순서와 호출 시점
- ✓ Phase 2 (Failover) 의 "RDS lag=0 확인 → DB 격리 → `terraform apply` (TG 교체) → ASG 대기 → `smoke test`" 안전장치 흐름 설계
- ✓ Phase 3 (Failback) 의 "Jenkins 가 RDS 직접 `mysqldump` → Ansible `site.yml` 재구축 → `ansible db_failback.yml restore` → `terraform apply pilot-light`" 흐름 설계
- ✓ tfvars **layered** 구조 활용 결정 (`pilot-light.tfvars` vs `dr-active.tfvars` 두 파일을 시나리오별로 갈라끼우는 전략)
- ✓ Pipeline 의 `parameters { choice() }` 분기 — 단일 Jenkinsfile 로 3 시나리오 처리하는 구조 결정

본인이 직접 **owned** 하지 않은 것:

- ✗ Terraform 모듈 코드 작성 (팀 + AI 협업)
- ✗ 4 모듈 분리 (`networking` / `database` / `compute` / `monitoring`) 의 결정 자체
- ✗ S3 + DynamoDB lock backend 셋업

1-2. 핵심 framing 멘트 (외워서 자연스럽게)

"DR 시스템에서 제일 중요한 건 **자원을 만드는 것보다 언제, 어떤 순서로, 무엇을 점검하면서 만드는지**입니다. Terraform 모듈은 자원 정의서고, Jenkins Pipeline 은 그 정의서를 어떤 순서로 펼칠지의 시나리오. 제 영역은 후자에 가깝습니다."

→ 이 한 문장이 본인 기여의 핵심 framing. 외울 것.

1-3. Pre-Sales 톤 보강 멘트

"코드 라인 수가 아니라 시스템 흐름 설계, 안전장치 결정, 그리고 검증·판단·채택의 *quality* 가 제 기여라고 봅니다. AI 시대의 엔지니어 기여는 손으로 친 코드량 보다 의사결정 *quality* 로 판단되는 영역이 점점 커지고 있고, 이 프로젝트가 그 예시 중 하나라고 생각해요."

→ 트러블슈팅 슬라이드 [39] 의 "AI 협업 패턴" narrative 와 정합. Closing 슬라이드 [40] 의 톤과도 자연 연결.

2. Q&A 대비 — Terraform 관련 질문 5종

Q1. "Terraform 코드 직접 보여줄 수 있나요?"

답변 멘트:

"Terraform 모듈 코드 자체는 슬라이드 15 (Terraform 담당 팀원) 에서 다뤘습니다. 제 슬라이드는 그 모듈을 Pipeline 안에서 어떻게 호출하는지 위주로 구성했어요. Jenkinsfile 의 `terraform apply -var-file=dr-active.tfvars` 한 줄이 ALB Listener Rule 교체 + ASG 0→2 scale up 까지 다 수행합니다. 호출 한 줄이 어떤 자원 변화를 만드느냐가 제 슬라이드 [38] 의 메시지에요."

→ 자연스럽게 팀원 슬라이드 referrer + 본인 영역 확정.

Q2. "Terraform 부분에서 본인이 직접 결정한 건 뭐예요?"

답변 멘트 (구체적인 본인 결정 3가지):

"세 가지가 제 결정입니다. (1) tfvars 를 평상시 (pilot-light) / Failover (dr-active) 두 파일로 layered 하게 두고 시나리오별로 갈아끼우는 전략. (2) terraform apply 호출 후 RDS lag=0 대기 / DB EC2 격리 / smoke test 같은 안전장치 stage 를 Pipeline 에 끼우는 흐름 설계. (3) Phase 3 Failback 에서 Jenkins 가 양쪽 네트워크에 접근 가능한 유일한 노드라는 점을 활용해 RDS 직접 mysqldump 하는 경로 설계 — 이걸 cascade 정합성 보장의 핵심이에요."

→ Concrete + 본인 영역 명확. 청중에게 "아, 이 사람은 Pipeline 흐름 설계에 집중했구나" 인상.

Q3. "Terraform 직접 안 짰으면 본인 기여가 뭔가요?" (까칠한 톤)

답변 멘트 (Pre-Sales 톤 정수, 외우기):

"AI 시대의 엔지니어 기여를 어떻게 정의하느냐의 질문이라 생각합니다. 제 기여는 세 가지예요 — (1) 시스템 흐름 설계, (2) 안전장치 결정, (3) 검증·판단·채택. 모듈 코드는 AI 와 팀이 함께 작성했지만, 그걸 어떤 순서로 호출 할지, 어떤 안전장치를 끼울지, 각 단계의 결과를 검증하고 다음 단계로 넘길지를 결정한 건 제 영역입니다. 코드 라인 수가 아니라 의사결정 quality 가 제 기여라고 봅니다."

→ 정직하면서도 강한 framing. 까칠한 질문도 받아칠 수 있음.

Q4. "AI 가 다 해줬다면 본인 학습은 뭐예요?"

답변 멘트:

"AI 가 답을 빨리 줘도, 그 답이 맞는지 판단하는 건 결국 사람의 영역입니다. 예를 들어 Tailscale ProxyJump 트러블슈팅 (트러블 narrative 1) 에서, AI 가 Subnet Router 우회를 제안했지만 그게 우리 환경에 맞는지 / 다른 부작용 없는지 검증한 건 제가 했어요. AI 시대 학습은 AI 가 못 주는 것 을 학습하는 영역으로 옮겨갔다고 봅니다 — 의문 던지기, 검증하기, 트레이드오프 판단하기."

→ 트러블슈팅 narrative 와 자연 연결. 본인이 미리 [39] 슬라이드 통해 이 메시지 예고했으니 강화 효과.

Q5. "Terraform 모듈 구조 (4 modules) 의 장단점 설명해주세요" (Terraform 깊은 질문)

답변 멘트 (정직하게 모르는 영역 인정):

"모듈 구조 설계는 Terraform 담당 팀원이 깊게 다뤘습니다. 슬라이드 15 의 Phase 2 부분이 그 영역이에요. 제 영역에서 답할 수 있는 부분은 — 4 모듈로 분리된 덕분에 Pipeline 에서 (compute / database / network 등) 시나리오별로 다른 모듈만 영향 주는 게 가능했다는 점. 예를 들어 Failover 시 networking 모듈은 건드리지 않고 compute 의 ALB Listener Rule 만 교체하는 식이에요."

→ "모르는 건 모른다" 정직 + "내 영역에서 알 수 있는 부분만 답". 신뢰 ↑.

3. Phase 이름 충돌 — 발표 중 disambiguate

3-1. 본인 슬라이드는 "시나리오" 로 통일 (확인)

Before (Phase)	After (시나리오)
Phase 1: 앱 배포	시나리오 1: 평상시 배포
Phase 2: Failover	시나리오 2: Failover (재해 전환)
Phase 3: Failback	시나리오 3: Failback (복구 복귀)

→ 슬라이드 위 텍스트, 본인 발표 중 구두 모두 "시나리오" 로 통일.

3-2. 발표 진입 시 1회 disambiguate (선택)

본인 파트 [33] 진입 시 한 번 명시 (옵션 — 시간 여유 있으면):

"잠깐 용어 정리만 하고 시작하겠습니다. 슬라이드 15 (Terraform 팀원) 에서 말씀하신 Phase 1-3.5 는 프로젝트 빌드 단계 — 자원을 차례로 만들어간 단계입니다. 제 발표에서 등장하는 시나리오 1/2/3 는 Jenkins 가 운영 중에 처리하는 다른 namespace 입니다."

→ 청중 혼란 즉시 차단. 단, 시간 빠듯하면 생략 — 이미 슬라이드에서 "시나리오" 라 적혀 있으면 자연스럽게 구분됨.

4. 본인이 owned 한 영역 — 자신 있게 말할 것 (정리)

발표 중 "이건 제가 했습니다" 가 자연스러운 영역. Q&A 깊은 질문 와도 답변 가능.

4-1. 기획 / 아키텍처

- 온프레미스 환경 셋업 + 네트워크 topology 구상
- 프로젝트 주제 + 전체 아키텍처 기획 (Hybrid DR with Pilot Light 패턴 선정)
- HAProxy-only Tailscale Subnet Router 패턴 결정 (slide [37] 의 핵심)

4-2. CI/CD 오케스트레이션 (Task 이름 = 본인 chapter 이름)

- 단일 Jenkinsfile 의 `parameters { choice() }` 3 시나리오 분기 구조
- 각 시나리오별 stage 설계 (Phase 2 = 6 stages, Phase 3 = 6 stages)
- 각 stage 의 안전장치 (lag=0 polling, DB 격리, smoke test 등)
- Phase 3 의 mysqldump 경로 설계 (Jenkins = 양 네트워크 접근 가능한 유일 노드)

4-3. Ansible (Phase 3 Failback 위주)

- `site.yml` 풀셋업 playbook 의 onprem 환경 재구축 흐름
- `db_failback.yml` 의 RDS → onprem master 정합성 복원 흐름
- 6 roles 중 본인 손댄 부분 (어느 role 인지 본인이 더 정확히 기억할 것)
- 역등성 활용 — `site.yml` 의 일부 role 이 `webwas.yml` 에 그대로 재사용되는 elegance ([35] 슬라이드 메시지)

4-4. AI 협업 패턴 ([39] 트러블슈팅 narrative)

- Tailscale ProxyJump 디버깅 — "본인 첫 의심 + AI 진단 chain" 패턴 정수
- IAM Bootstrap 사전 분리 — 본인 사전 설계 (admin IAM User 따로 보유) 가 답이었던 사례
- Ansible `ansible_user` 누락 — "근데 왜 jenkins?" 첫 의문이 답을 부른 사례

5. 본인이 owned 하지 않은 영역 — 발표 중 referrer 처리

"이건 팀원 부분입니다" 자연스럽게 referrer. 청중 혼란 X.

영역	담당 팀원	본인 멘트
Terraform 모듈 코드	Terraform 담당 (슬라이드 15)	"모듈 구조는 슬라이드 15 에서 다뤘고, 제 슬라이드는 호출 흐름 위주입니다"
GTID Cascade 복제	DB 담당 (슬라이드 26-29)	"방금 DB 팀원이 보여드린 cascade 위에서 제 Pipeline 이 동작합니다" (bridge 멘트)
CloudWatch alarm 설계	Monitoring 담당 (슬라이드 30-31)	(본인 파트에서 언급 안 해도 됨)
Spring Boot DR 화면	(담당 팀원, 슬라이드 22)	(본인 파트에서 언급 안 해도 됨)

→ **bridge 멘트 활용**: DB 팀원 발표 직후 본인 [37] (Tailscale) 진입 시 "방금 DB 팀원이 보여드린 cascade 가 동작하려면, AWS DB-EC2 가 온프레미스 master 의 binlog 를 읽어야 합니다. 그 라우팅을 만든 게 바로 제가 결정한 Tailscale 패턴 이예요. 여기서부터 제 부분이 시작됩니다." → 팀 발표 흐름 매끄러워짐 + 협업 신호.

6. 발표 직전 체크리스트

- ☐ "Terraform 모듈은 자원 정의서, Pipeline 은 호출 시나리오" framing 멘트 외움
- ☐ "AI 시대 엔지니어 기여 = 의사결정 quality" 멘트 외움
- ☐ 본인 슬라이드의 모든 "Phase" → "시나리오" 갱신 확인
- ☐ [33] 4박스 안 의 카테고리 4개 ("Jenkinsfile / Ansible / Tailscale / 아키텍처 기획") 외움
- ☐ DB 팀원 → 본인 [37] bridge 멘트 외움
- ☐ Q&A 5종 답변 시뮬레이션 (Q3 Pre-Sales 정수 멘트 특히 외우기)

7. 변경 사항이 생기면 여기 추가

이 문서는 발표 직전까지 가다듬는 살아있는 노트. 새로 결정된 멘트 / 더 좋은 framing 발견 시 여기 갱신.

- (TBD)

8. 학습용 공식 참고 자료 — Tailscale Subnet Router / Hybrid 패턴

이 섹션 사용: 5/4 (토) ~ 5/5 (일) 학습 시 정독. 발표 멘트 "*Tailscale 공식 reference architecture 가 권장하는 패턴*" 의 출처 확인 + Q&A 깊은 질문 대비.

8-1. ★ 가장 결정적인 근거 (외워두면 Q&A 강력)

AWS Reference Architecture ← 본인 멘트의 출처 <https://tailscale.com/docs/reference/reference-architectures/aws>

→ Tailscale 이 "*reference architecture*" 라는 용어로 공식 권장하는 AWS 통합 가이드. "*우리가 발명한 게 아니라 권장 구조 적용 사례*" 멘트의 직접 backup.

Connect to an AWS VPC using subnet routes <https://tailscale.com/docs/install/cloud/aws>

→ Subnet Router 로 AWS VPC 전체를 Tailscale 에 포함시키는 방법. **핵심 인용 문구:**

"agent-to-agent connectivity for connecting to static resources like EC2 instances ... and IP-based connectivity with a Tailscale subnet router to connect to managed AWS resources such as Amazon RDS or Amazon Redshift (recommended where you cannot run Tailscale on the resource or want to expose an existing subnet or services in a VPC)."

→ "*agent* 깔 수 있는 자원은 직접 깔고, 못 까는 자원 (RDS) 은 Subnet Router 로" 가 공식 권장 분기임을 보여주는 결정적 문장. 본인 발표의 "*흔한 방식 vs 우리 선택*" narrative 와 정확히 맞아떨어짐.

8-2. RDS 관련 결정적 KB

Access AWS RDS privately using Tailscale <https://tailscale.com/kb/1141/aws-rds>

→ "*RDS 는 Subnet Router 로 접근*" 이 별도 KB 까지 만들어진 공식 권장 패턴. "*RDS 만 별도 우회로 만들었을 거*" narrative 의 정직함 backup.

8-3. 기초 개념

Subnet routers (개념 문서) <https://tailscale.com/docs/features/subnet-routers>

→ Subnet Router 의 정의: "*don't or can't run the Tailscale client*" 자원을 위한 gateway 패턴. 본인 설계의 메커니즘 자체.

Configure a subnet router (실습용) <https://tailscale.com/docs/features/subnet-routers/how-to/setup>

→ `tailscale up --advertise-routes=...` 옵션 가이드. 본인이 실제 사용한 명령의 공식 reference.

8-4. Q&A 깊은 질문 대비

Set up high availability ← "*HAProxy SPOF 어쩔 거냐*" 질문 대비 <https://tailscale.com/docs/how-to/set-up-high-availability>

→ **핵심 인용:** "*multiple subnet routers can be deployed across multiple availability zones and configured to advertise the same routes to achieve high availability failover*". 본인 [39] (Closing) 의 "*실 사용 시 HAProxy 다중화*" 답변의 공식 backup.

Site-to-site networking ← Tailscale hybrid 패턴 카탈로그 <https://tailscale.com/docs/features/site-to-site>

→ onprem ↔ cloud 패턴들 정리. 본인 패턴이 그 카탈로그의 표준 적용임을 확인.

8-5. 학습 체크리스트 (5/4-5/5)

- ☐ AWS Reference Architecture 정독 (8-1) — "reference" 단어 빈도 + 권장 표현 정확히 캡처
- ☐ AWS VPC subnet routes (8-1) — agent-to-agent vs subnet router 분기 기준 명확히 이해
- ☐ AWS RDS KB (8-2) — "RDS 만 다른 길" 의 흔한 함정 영문 표현 1-2개 찾기
- ☐ HA 문서 (8-4) — multiple subnet routers + advertise same routes 메커니즘 이해
- ☐ **본인 멘트와 공식 문구 정합 점검** — 발표 중 "공식 권장 패턴" 이라 했는데 청중이 의심 표정 지을 때 "바로 이 문서입니다" 답할 수 있도록 URL 1-2개는 외워두기 (특히 8-1)

9. [36] Ansible 슬라이드 — 핵심 개념 학습 노트

이 섹션 사용: 5/4 (토) 여행연습 시 정독. 본인이 직접 catch 한 개념 모순 + Spring Boot 단일 jar narrative 의 정확한 framing.

9-1. "코드 중복 0% vs 같은 코드 재사용" — 모순 같지만 같은 의미 ★

본인이 발표 준비 중 catch 한 의문: "코드 중복이 0% 인데 같은 코드를 두 시나리오에서 재사용한다? 말이 잘못된 거 아닌가?"

답: 모순 아닙니다. 같은 의미의 두 표현입니다.

개념	의미
코드 중복 100%	같은 로직을 두 번 적어둠 (두 파일에 같은 코드 사본 2개)
코드 중복 0%	같은 로직을 한 번만 적어둠 + 두 곳에서 호출 (사본 1개를 reference)

→ "같은 코드 재사용" = "한 번 적은 걸 두 곳에서 가져다 씀" = "중복 적지 않음" = **코드 중복 0%**

구체적 예시:

- `springboot` role 의 코드 = `Ansible/roles/springboot/` 폴더에 **한 번만 작성**
- `site.yml` 에서 - `springboot` 호출 (참조)
- `webwas.yml` 에서도 - `springboot` 호출 (같은 폴더 참조)
- → 코드는 **한 곳에만 존재**, 두 시나리오가 공유

만약 중복이었다면: `site.yml` 용 `springboot-for-site` 폴더 + `webwas.yml` 용 `springboot-for-webwas` 폴더 = 똑같은 코드를 두 번 적어둔 상태 (중복).

발표 중 입으로 풀 멘트:

"role 코드는 한 곳에만 두고, 두 시나리오에서 동일하게 호출 — 같은 로직을 두 번 안 적어도 되니 코드 중복 0% 예요."

→ 슬라이드 우측 텍스트는 그대로 ("→ 코드 중복 0%") 두고 입으로 풀어주면 청중도 이해 ↑

9-2. Spring Boot 단일 jar narrative — 풀어 설명

원래 narrative 한 줄 요약:

"WEB-WAS 의 서버 데몬을 Nginx + Tomcat 에서 Spring Boot 단일 jar 로 바꾸면서, 시나리오 3 의 Ansible 코드가 시나리오 1 에도 재사용 가능해졌다 — 기술 선택이 다른 기술 선택을 단순화한 케이스."

풀어서 이해:

Nginx + Tomcat 였다면 (Java EE 전통 방식)

- Nginx (정적 파일 서빙) + Tomcat (Java 앱 실행) **두 개 서비스 분리**
- 앱 갱신 시 절차:
 1. WAR 파일 빌드
 2. Tomcat 에 WAR 배포 (Tomcat manager API 호출 또는 webapps/ 폴더 교체)
 3. Tomcat 재시작
 4. Nginx 설정 점검 (proxy_pass 경로 등)
 5. 헬스체크
- Ansible 코드 **두 종류 필요**: `nginx` role + `tomcat` role
- WAR 배포용 Ansible task = **여러 단계** (WAR 업로드 / Tomcat 제어 / 재시작 / 검증 ...)

Spring Boot 단일 jar 로 바꾸면

- **jar 안에 임베디드 Tomcat 포함**, 의존성 다 들어 있음 (self-contained)
- 앱 갱신 = `.jar` 파일 하나 교체 + `systemctl restart` 끝
- Ansible 코드: `springboot` role **하나면 충분** (jar copy + service restart)
- WAR 배포 / Nginx 설정 점검 / Tomcat manager 같은 단계 **모두 불필요**

연쇄 효과 — "기술 선택이 다른 기술 선택을 단순화" 의 도식

```

WEB-WAS 기술 선택 (Spring Boot 단일 jar)
  ↓ 영향
앱 자체 단순화 (jar 한 개)
  ↓ 영향
배포 절차 단순화 (jar 교체 + 재시작)
  ↓ 영향
Ansible role 단순화 (springboot role 하나면 충분)
  ↓ 영향
시나리오 3 의 role 코드를 시나리오 1 에서 그대로 재사용 가능
  ↓ 영향
코드 중복 0% + 운영자 부담 ↓ + 앱 최신화 자동화 단순
  
```

→ 한 영역 (서버 데몬) 의 단순화 결정이, 그 위에서 동작하는 자동화 도구 (Ansible) 까지 자연스럽게 단순하게 만든 **연쇄 효과**.

→ Pre-Sales 톤으로 연결하면: "좋은 기술 선택은 단독으로 작용하지 않고 인접 영역까지 자연스럽게 단순화한다" 의 case study.

9-3. 매끄럽게 다듬은 [36] Speaker Note (최종 — 슬라이드 layout 반영)

슬라이드 layout: 좌측 Role 매트릭스 + 우상단 webwas.yml 코드 + 우하단 site.yml 코드 + 하단 핵심 포인트 박스.

(슬라이드 진입) 앞 슬라이드의 Tailscale 라우팅 위에서 동작하는 자동화 도구 — Ansible 을 저희 시스템에서 어떻게 사용했는지 보여드리겠습니다.

(좌측 매트릭스 가리키며) 6 개 role 을 호스트 그룹별로 나눠서 적용한 매트릭스입니다. **common** 과 **cloudwatch-agent** 는 모든 호스트, **tailscale** 은 HAProxy 만 (앞 슬라이드의 패턴과 일치), 나머지는 각 역할 호스트에만 — 책임을 잘게 나눈 구조예요.

(우상단 webwas.yml 가리키며) 위쪽이 시나리오 1 의 **webwas.yml** — 단 5줄, 3 roles 만 (**common** / **springboot** / **cloudwatch-agent**). 평상시 앱 갱신 시 jar 만 갈아끼우는 단축 경로입니다.

(우하단 site.yml 가리키며) 아래쪽이 시나리오 3 의 **site.yml** — 18줄, 6 roles. 시나리오 3 (Failback) 에서 온프레임 환경을 풀셋업할 때 사용해요.

자세히 보시면 webwas.yml 의 3 roles 가 모두 site.yml 안에 그대로 들어 있습니다. **role 코드는 한 곳에만 두고, 두 시나리오에서 동일하게 호출 — 같은 로직을 두 번 안 적어도 되니 코드 중복 0%** 입니다.

(역등성 의미 짧게) 역등성이란 같은 playbook 을 100번 실행해도 100번 모두 같은 결과를 보장한다는 의미예요. 원하는 상태를 선언하면 Ansible 이 현재 상태와 비교해서 차이만 적용합니다. DR 에서 이게 중요한 이유는, **부분 실패 후 다시 돌려도 안전하다는 점**입니다.

(narrative) 처음엔 시나리오 3 (Failback) 의 자동 복구를 위해 Ansible 을 도입했습니다.

그러나 WEB-WAS 를 **Nginx + Tomcat 에서 Spring Boot 단일 jar 로** 바꾸면서, 앱 갱신이 jar 파일 하나 교체 + 서비스 재시작만으로 단순해졌어요.

원래 Nginx + Tomcat 였다면 앱 갱신 시 WAR 배포 / Tomcat 재시작 / Nginx 설정 점검 등 여러 단계 코드가 필요했을 텐데, Spring Boot 단일 jar 는 jar 한 개만 다루면 되니까 자동화 코드가 짧아져요.

그 결과 시나리오 3 을 위해 짰던 Ansible 코드가 시나리오 1 앱 업데이트에도 그대로 재사용이 가능해진 겁니다. 즉, **기술 선택이 다른 기술 선택을 단순화한 케이스**입니다.

→ 예상 시간: 약 1분

9-4. Q&A 대비 — Ansible / Spring Boot 관련 추가

Q. "왜 Spring Boot 단일 jar 로 바꿨나요? Nginx + Tomcat 도 정상 동작하는 구성인데."

답변 멘트:

"세 가지 이유였어요. 첫째, **단일 jar 로 빌드 / 배포가 간편** — CI/CD 단순화. WAR + Tomcat manager 호출 같은 다단계 절차가 필요 없습니다. 둘째, **임베디드 Tomcat** 으로 별도 WAS 설정이 필요 없어요. 의존성과 서버가 jar 하나에 다 들어 있어 self-contained 입니다. 셋째, **jar 하나만 갱신하면 되니 자동화 코드도 단순해진다는 점** — 이게 발표에서 보여드린 'Ansible role 재사용' 의 enabler 가 됐어요. 한 영역 단순화가 인접 영역까지 단순화한 케이스예요."

→ Pre-Sales 톤으로 연결: "기술 선택이 단독으로 작용하지 않고 시스템 전체에 영향" 메시지 강조.

Q. "역등성 보장 안 되는 task 도 있나요?"

답변 멘트:

"있습니다. shell 모듈로 현재 상태 무시하고 항상 실행하는 task — 이걸 Ansible idiom 위반이라 review 단계에서 걸려야 합니다. 우리 프로젝트는 검증 task 외에는 모두 멍등성 모듈을 사용했어요."

Q. "Configuration drift 가 뭐예요?"**답변 멘트:**

"운영 중 사람이 수동으로 서버 만지면 코드 정의와 실제 상태가 어긋나는 현상입니다. Ansible 을 주기적으로 재 실행하면 자동으로 원래 상태로 되돌려놓음 — drift 방어. snowflake 서버 (눈송이처럼 다 다른 서버) 가 안 생기게 하는 표준 패턴이에요."

Q. "Ansible 말고 Bash script 로도 되지 않나요?"**답변 멘트:**

"가능하지만 멍등성을 직접 구현해야 하고, 호스트 여러 대 관리 시 ssh 루프를 직접 돌려야 합니다. 변수 관리, 템플릿 처리, 에러 핸들링 모두 직접 — 결국 Ansible 을 다시 만들게 됩니다. YAML 한 번 쓰면 끝나는 걸 굳이 안 해도 되죠."

Q. "site.yml 과 webwas.yml 은 정확히 어떻게 다른가요?"**답변 멘트:**

"**site.yml** 은 6 roles 모두 적용 — 온프레미 환경 풀셋업 (HAProxy / Tailscale / MySQL / SpringBoot / common / cloudwatch-agent). 시나리오 3 (Failback) 에서 온프레미 환경을 처음부터 재구축할 때 사용합니다. **webwas.yml** 은 3 roles 만 — **common**, **springboot**, **cloudwatch-agent**. WEB-WAS 호스트만 대상으로 jar 갱신 + 서비스 재시작. 시나리오 1 의 평상시 앱 배포 전용입니다. 두 playbook 이 같은 **springboot** role 을 호출 — 그래서 '같은 코드, 다른 시나리오' 의 의미예요."

9-5. 5/4 여행연습 체크리스트 — Ansible 슬라이드 [36]

- ☐ "코드 중복 0% vs 재사용" 은 모순 아니라 같은 의미임을 자신 있게 설명
- ☐ Nginx + Tomcat → Spring Boot 단일 jar narrative 자연스럽게 풀기 (위 9-2 도식 외워두기)
- ☐ "기술 선택이 다른 기술 선택을 단순화" 의 연쇄 효과 한 문장으로 요약 가능
- ☐ "왜 Spring Boot 로 바꿨나?" Q&A 3가지 이유 (빌드 간편 / 임베디드 / 자동화 단순) 외움
- ☐ 멍등성 정의 한 문장 ("같은 playbook 100번 실행해도 100번 모두 같은 결과") 외움
- ☐ DR 에서 멍등성이 중요한 이유 ("부분 실패 후 다시 돌려도 안전") 외움
- ☐ site.yml vs webwas.yml 차이 (6 roles vs 3 roles) 정확히 설명 가능

10. [38] 트러블슈팅 슬라이드 — narrative + 톤 노트

이 섹션 사용: 5/4 (토) 여행연습 시 정독. 본인 파트의 highlight 슬라이드 — narrative 강도 + 겸손 톤 균형이 핵심.

10-1. 슬라이드 layout

- 3-Column 표 (Network 녹색 / Cloud Security 주황 / Automation 파랑/브랜드)

- 각 컬럼 4행: 헤더 / 트러블 제목 / 증상 / 해결 + 🌀 결과
- 표 하단 strip (전체 폭): "세 트러블의 공통점 — 본인 의문이 시작점 · AI 진단이 가속 · 본인 채택이 마무리"
- 시간: ~3분 (가장 김)

🌀 **행 (4행) 텍스트** (옵션 1 — 자연스러운 톤 적용):

Network	Cloud Security	Automation
→ 더 좋은 아키텍처로 이어진 트러블	→ 사전 설계가 답이 된 케이스	→ 첫 질문이 답을 빨리 부른 케이스
→ "~한 사례 / ~한 케이스" 서술형. "~해야 한다" 단언 X. "의사결정 매트릭스", "사전 설계의 가치", "디버깅 속도 결정" 같은 consulting-speak 회피.		

10-2. ★ 톤 원칙 — "단언" + "강제 lesson box" 회피

본인이 catch 한 두 가지 issue:

Issue 1 (5/4 오전): "AI 시대 엔지니어의 표준 협업 패턴" 같은 표현 → 본인이 그 "표준" 을 정의하는 듯한 단언 → 졸업생 톤과 안 맞음.

Issue 2 (5/4 오후): 각 narrative 마다 "여기서 배운 건..." 패턴 반복 → 학원 case study 보고서 톤. 실무진 입장에서 "교과서 정답 외워서 던지는 사람" 인상.

해결 — 다음 단어/패턴 회피:

❌ 회피	✅ 대체
"표준 협업 패턴"	"세 트러블의 공통점" / "이렇게 일하는 방식"
"AI 시대 엔지니어의"	"제가 익혀가고 있는" / "제가 배운 한 가지"
"이게 답이다"	"이렇게 했습니다" / "이런 식으로 배웠어요"
"= AI 시대 표준" (등호 + 단언)	"세 트러블의 공통점" (관찰)
"여기서 배운 건 ... 한다는 점입니다" (반복 패턴)	각 narrative 자연스럽게 마무리. 인위적 lesson box X
"의사결정 매트릭스 사전 제시"	"더 좋은 설계로 이어진 사례" (story 요약)
"사전 설계의 가치"	"우연이 아니라 사전 설계였기 때문에 막힘 없이 풀 수 있었습니다" (경험 진술)
"디버깅 속도를 결정"	"솔직히 든 생각이 ..." (개인 reflection)

Pre-Sales 톤의 본질 = "내가 어떻게 일했는지" 를 보여주는 것 (implicit), "보편 진리는 이거다" 단언 (explicit) X.

- ✅ "이 트러블에서 이렇게 했고, 풀고 나니 이런 게 보였습니다" 정도
- ❌ "이게 표준이고 모두가 이래야 한다"

→ 청중이 "이 사람 일 잘하네 + 겸손하네" 로 받아들임. "건방지네 + 분수 모르네" X.

10-3. 매끄럽게 다듬은 [38] Speaker Note (최종 — 자연스러운 톤)

(슬라이드 진입 + 3 trouble 소개) 지금부터 세 가지 트러블 슈팅 사례를 보여드리겠습니다. Network, Cloud Security, Automation — 서로 다른 3 layer 에 걸친 장애 상황이었습니다.

(① Network — Tailscale ProxyJump, ~55초) 첫 번째는 Tailscale ProxyJump 장애 상황입니다.

시나리오 1 첫 빌드에서, ProxyJump 조합만 실패했습니다. TCP 연결, SSH banner, ControlMaster, **-W** 수동 — 4 레이어 격리 검증을 다 해봐도, 조합만 통과가 안 됐습니다.

3시간 정도 헤매다가 AI 에게 질문했고, Subnet Router 우회를 제안받아 10분 만에 풀렸습니다.

근데 이 트러블이 단순히 우회로 끝난 게 아니라, 결과적으로 앞에서 보여드린 **HAProxy-only Tailscale 아키텍처의 출발점**이 됐습니다. 트러블 슈팅 과정에서 더 좋은 설계로 이어진 사례라고 생각합니다.

(② Cloud Security — IAM Bootstrap 역설, ~45초) 두 번째는 IAM Bootstrap 역설입니다.

Jenkins 가 자체 terraform 을 실행하려면 IAM 권한이 필요한데, 그 권한을 추가하려면 **terraform apply** 가 필요한 — Reddit 같은 IT 커뮤니티에서 흔히 "**닭과 달걀 역설**" 이라고 비유하는 상황이었습니다.

다행히 처음부터 운영용과 bootstrap 용 (CLI 접근만 가능한) IAM User 를 별도로 분리해놔던 게 답이 됐습니다. 우연이 아니라 사전 설계였기 때문에, 막상 트러블이 닥쳤을 때 막힘 없이 풀 수 있었습니다.

(③ Automation — Ansible ansible_user 누락, ~50초) 세 번째는 Ansible **ansible_user** 누락입니다.

시나리오 3 빌드 중에 **jenkins@호스트: Permission denied** 메시지가 떴습니다. Console output 을 확인해보니 사용자명이 jenkins 로 찍혀 있었습니다.

"Ansible 을 실행했는데, 왜 사용자명이 jenkins 로 잡혀 있는 걸까?" 라는 의문을 던졌고, 이걸 AI agent 에게 질문했습니다. 받은 진단 chain 을 따라가보니 **ansible_user** 변수 누락이 원인이었고, **group_vars/all.yml** 을 수정해서 해결했습니다.

솔직히 이 사례에서 든 생각이, 첫 질문을 정확하게 던지는 게 디버깅의 절반이라는 점이었습니다. AI 가 답을 빠르게 줘도, 어디서부터 물어볼지는 결국 사람이 정하니까요.

(Closing — 자연스러운 톤, 슬라이드 하단 strip 가리키며) 세 가지가 다 다른 영역이었지만, 풀어가는 과정은 비슷했습니다.

처음 막혔을 때 의문을 정확하게 던지는 것, AI 에게 물어보고 답변을 검증하는 것, 그리고 적용하는 것.

졸업 프로젝트를 진행하면서, 이런 식으로 일하는 법을 익혀가고 있습니다.

→ **예상 시간:** 약 2:50 ~ 3:10

무엇이 바뀌었나 (5/4 오후 다듬음)

이전 (lesson box 강제)	이후 (자연스러운 narrative)
"여기서 배운 건... 한다는 점입니다" (3번 반복)	각 narrative 자연스럽게 마무리
"의사결정 매트릭스 사전 제시"	"트러블 슈팅 과정에서 더 좋은 설계로 이어진 사례"
"사전 설계의 가치"	"우연이 아니라 사전 설계였기 때문에 막힘 없이 풀 수 있었습니다"

이전 (lesson box 강제)**이후 (자연스러운 narrative)**

"정확한 첫 의심이 디버깅 속도를 결정"

"솔직히 든 생각이, 첫 질문을 정확하게 던지는 게 디버깅의 절반"

"AI 시대 엔지니어의 표준 협업 패턴"

"이런 식으로 일하는 법을 익혀가고 있습니다" (학습자 톤)

10-4. Q&A 대비 — 트러블슈팅 관련**Q. "AI 가 답을 다 줬다면 본인 기여는 뭐예요?"**

답변 멘트 (겸손 + 정직 톤):

"AI 는 가설 후보를 빠르게 제공해줬고, 저는 그걸 검증·판단·채택했어요. 첫 의문을 던진 것도 저였습니다. 같이 결론에 도달한 거지, AI 가 혼자 답을 준 건 아니에요."

→ 절대 "AI 가 답을 준 게 아니라" 같은 단언형 X. "AI 와 함께", "같이" 같은 협업 톤 사용.

Q. "가장 어려웠던 디버깅은 뭐였나요?"

답변 멘트:

"슬라이드의 세 사례 외에 백업으로 GTID Multi-Node Align 트러블도 있었어요. 슬라이드에는 안 넣었지만 자세히 풀어드릴 수 있습니다." → [troubleshooting-narratives.md](#) 의 GTID 백업 narrative 활용

Q. "AI 사용에 대한 본인 입장은?"

답변 멘트 (겸손하게):

"AI 는 가설을 빠르게 보여주는 도구로 정말 강력했어요. 다만 그 가설이 우리 환경에 맞는·다른 부작용이 없는지 검증하는 건 결국 사람의 영역이라고 느꼈습니다. 저는 아직 익혀가는 중이고요."

→ "이게 답이다" 보다 "제가 느낀 건", "익혀가는 중" 같은 겸손 표현.

10-5. 5/4 예행연습 체크리스트 — 트러블슈팅 슬라이드 [38]

- ☐ 3 narrative 의 시간 배분 자연스럽게 (각 ~50초)
- ☐ "여기서 배운 건..." 패턴 입에서 안 나오는지 점검 (강제 lesson box 회피)
- ☐ "표준", "AI 시대 엔지니어의", "의사결정 매트릭스 사전 제시" 같은 consulting-speak 단어 회피
- ☐ "이런 식으로 일하는 법을 익혀가고 있습니다" 마무리 톤 자연스럽게 (학습자 정체성 유지)
- ☐ Tailscale narrative → 앞 슬라이드 [33] HAProxy-only 아키텍처의 출발점이라는 점 자연스럽게 연결
- ☐ IAM narrative → "우연이 아니라 사전 설계였기 때문에 막힘 없이 풀 수 있었습니다" 자연스러운 진술
- ☐ Ansible narrative → "왜 사용자명이 jenkins 로 잡혀 있는 걸까?" 의문 + "솔직히 든 생각이..." 개인 reflection
- ☐ Closing → "세 가지가 다 다른 영역이었지만, 풀어가는 과정은 비슷했습니다" 자연 흐름
- ☐ Q&A "AI 가 답을 다 줬으면..." 답변 (겸손 톤) 외움
- ☐ 거울 보고 한 번 발표 — "건방진 톤" 안 나오는지 자가 점검